COMP1531
Software Engineering Fundamentals

Week 7

Full Stack - Auth

In this Lecture:

– – –

- What is Authentication?
- Storing User Credentials
- Registration Workflow
- Login Workflow
- Secure Password Handling

🔐 Authentication & Authorisation

– – –

Auth : A nickname in COMP1531 for two related, but distinct security concepts:

Authentication

→ The process of verifying **who** a user is
(e.g., login with email + password)

Authorisation

→ The process of determining **what** an authenticated user is allowed to do
(e.g., student vs. tutor vs. admin access on discourse forum)

🔐 Authentication

- What is the the most basic approach?

1️⃣ Registration Workflow

1. User provides email + password
2. System checks if user already exists
3. Store: { email: {password}}

2️⃣ Login Workflow

1. User enters credentials
2. System retrieves stored password
3. Comparison to verify match

🔐 Authentication

TS auth_simple.ts > ...

```ts
type Data = { users: { [email: string]: string };  };

const data: Data = {
  users: {},
};

function register(email: string, pw: string) {
  if (email in data.users) {
    return false;
  } else {
    data.users[email] = pw;
    return true;
  }
}

function login(email: string, pw: string) {
  if (email in data.users) {
    if (pw === data.users[email]) {
      return true;
    }
  }
  return false;
}
```

BUT ...

what's wrong with this?

🔐 Authentication

What's wrong with this?

We're storing peoples' passwords!

In this example, yes, it's just being stored in a variable in RAM, which is OK. But in reality, our "data" would be stored on a hard drive long term! Which is scary. How do we avoid this??

We need to 🦄 hide 🦄 the password

..what does that even mean..

🔐 Secure Password Storage - Hashing

— — —

Why not store plaintext passwords?

If leaked, attackers instantly access every account.

Hashing
A one-way cryptographic function that converts a password into a fixed-length value.

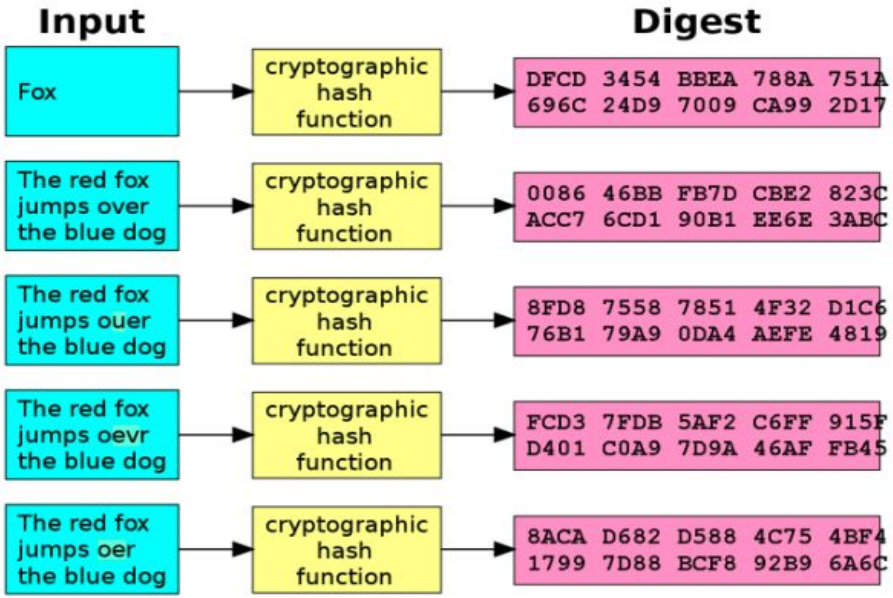
Example:

"abc123" → 2c26b46b68ffc68f...

Cannot be reversed back into the original password.

🔐 Secure Password Storage - Hashing

Hashing is the processes of taking plaintext information and concealing it by turning it into a seemingly random string of characters.



Source: Wikipedia

🔐 Hiding Information

Let's explore a hashing example together.

- We will use the **`crypto`** library. This is **built-in** to NodeJS so you don't need to `npm install` anything.

TS hash.ts > ...

```ts
import crypto from 'crypto';

function getHashOf(plaintext: string) {
  return crypto.createHash('sha256').update(plaintext).digest('hex');
}

const msg = 'secretPassword';
const hash = getHashOf(msg);

console.log(hash);

export { getHashOf }; // ignore this line
```

– – –

getHashOf converted “**secretPassword**” to

8c64be3db244091660a3b69ef548e3d43f9f945aaae78e1ff2de939cac1116ba

There is **no way** to convert hash to msg! Even if you know the entire method of how this conversion happens, it's not reasonably possible to reverse it.

🔐 Hashing

- How would we apply this to our authentication issue?

Our previous approach:

1️⃣ Registration Workflow

1. User provides email + password
2. System checks if user already exists
3. Store: { email: {password}}

2️⃣ Login Workflow

1. User enters credentials
2. System retrieves stored password
3. Comparison to verify match

🔐 Hashing

- How would we apply this to our authentication issue?

Improved Approach

1️⃣ Registration Workflow

1. User provides email + password
2. System checks if user already exists
3. **Hash** the password
4. Store: { email: {passwordHash}}

2️⃣ Login Workflow

1. User enters credentials
2. System retrieves stored hash
3. Recomputes hash using input password
4. Comparison to verify match

🔐 Hashing

How would we apply this to our authentication issue?

TS auth_simple.ts > ...

```ts
type Data = { users: { [email: string]: string };  };

const data: Data = {
  users: {},
};

function register(email: string, pw: string) {
  if (email in data.users) {
    return false;
  } else {
    data.users[email] = pw;
    return true;
  }
}

function login(email: string, pw: string) {
  if (email in data.users) {
    if (pw === data.users[email]) {
      return true;
    }
  }
  return false;
}
```

auth_simple.ts

🔐 Hashing

How would we apply this to our authentication issue?

```ts
type Data = {
  users: { [email: string]: string };
};

import { getHashOf } from './hash';

const data: Data = {
  users: {},
};

function register(email: string, pw: string) {
  if (email in data.users) {
    return false;
  } else {
    data.users[email] = getHashOf(pw);
    return true;
  }
}

function login(email: string, pw: string) {
  if (email in data.users) {
    if (getHashOf(pw) === data.users[email]) {
      return true;
    }
  }
  return false;
}
```

auth_fixed.ts

What is Encryption ?

🔐 Encryption - Protecting Data Confidentiality

Encryption transforms readable data (**plaintext**) into an unreadable form (**ciphertext**) so that only authorized parties can reverse it.

How It Works

```
Plaintext + Key → Encrypt → Ciphertext

Ciphertext + Key → Decrypt → Plaintext
```

➔ Reversible using the correct key
➔ Protects sensitive data **in transit and at rest**

Example usage: HTTPS (web security with TLS), Securing stored files, credit cards, Messaging apps (end-to-end encryption)

Encryption

TS encryption.ts > ...

```ts
import crypto from 'crypto';

const key = crypto.randomBytes(32);
const iv = crypto.randomBytes(16);

function encrypt(text: string) {
  const cipher = crypto.createCipheriv('aes-256-cbc', key, iv);
  return cipher.update(text, 'utf8', 'hex') + cipher.final('hex');
}

function decrypt(text: string) {
  const decipher = crypto.createDecipheriv('aes-256-cbc', key, iv);
  return decipher.update(text, 'hex', 'utf8') + decipher.final('utf8');
}
```

– – –

Remember: Encryption ≠ Password hashing

(Encryption can be reversed; hashes cannot)

Reversibility does make the hiding process finitely **less secure** (since a method to "unhide" the information exists). But it provides the convenience of being able to reverse it!

🔐 Encryption vs Hashing

Feature	Encryption	Hashing
Purpose	Protect confidentiality of data	Securely store & verify passwords
Reversible?	Yes - original data can be recovered with a **key**	No - one-way transformation
Output changes if same input used?	Not necessarily	No but - using *salts* make hashes output unique (extra security) **
Stored Data	Encrypted text + key management	Hash (+ salt)
Example Use Cases	Messages, files, HTTPS traffic	Password authentication

🔐 Managing User Tokens

User tokens often contain identity and important claims (session information) such as:

- `userId` - who am I?
- `role` - what am I? (student/admin)
- `permissions` - what can I do?
- `Expiry timestamp` - when does my session expire?

If a client can modify a token like:

```
{ "userId": 5, "role": "admin" , "permissions": ["read"],"exp": "2025-11-01T10:00:00Z"
}
```

… and the server **trusts** it blindly → Privilege Escalation Attack eg. the attacker becomes an admin.

**** Why tokens are sent in HTTP headers (not body)?**

🔐 Can we use Hashing for Tamper Proofing Token Data?

Let's explore two methods together to see if we can make our token data tamper proof:

- **Obfuscating** the session ID or user ID and doing server side lookup (what are problems with this?)

  Obfuscation = disguising data so it looks different or confusing, but can still be reversed or understood with some effort.

  - **Example:**

    ```
    token = base64(userId)
    ```

    - No mechanism for integrity check - server has no cryptographic way to tell the difference between original and modified tokens. Requires server-side lookup on every request.
- **"Signing"** our payload with a hash that helps us understand if someone has tampered with the payload.
  - **Example**

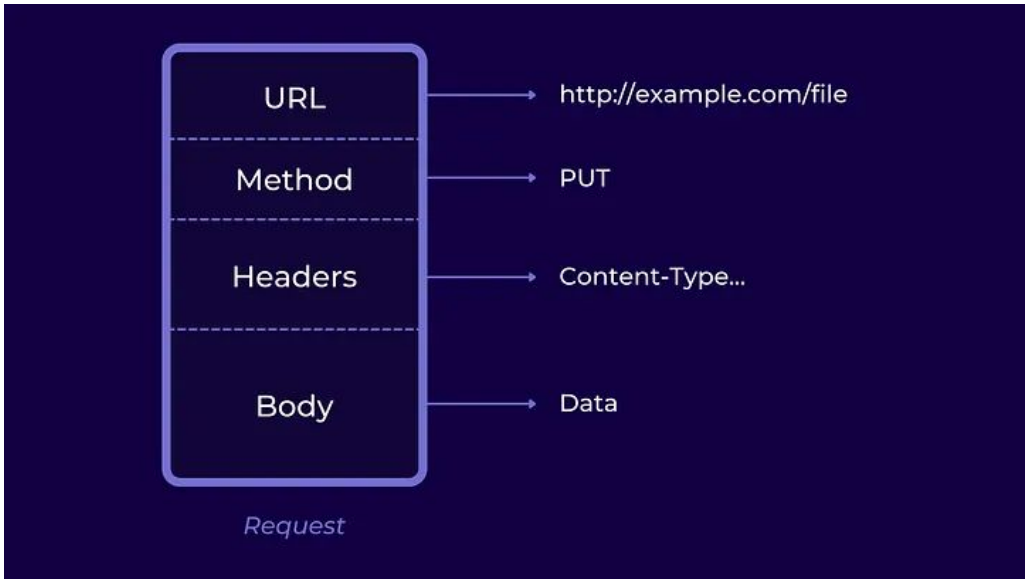
    ```
    payload = { userId: 123 }

    signature = HMAC(secret, payload)

    token = base64(payload + signature)
    ```

Benefits
✅ Server can detect changes → **tamper-evident**
✅ No DB lookup required to trust the data

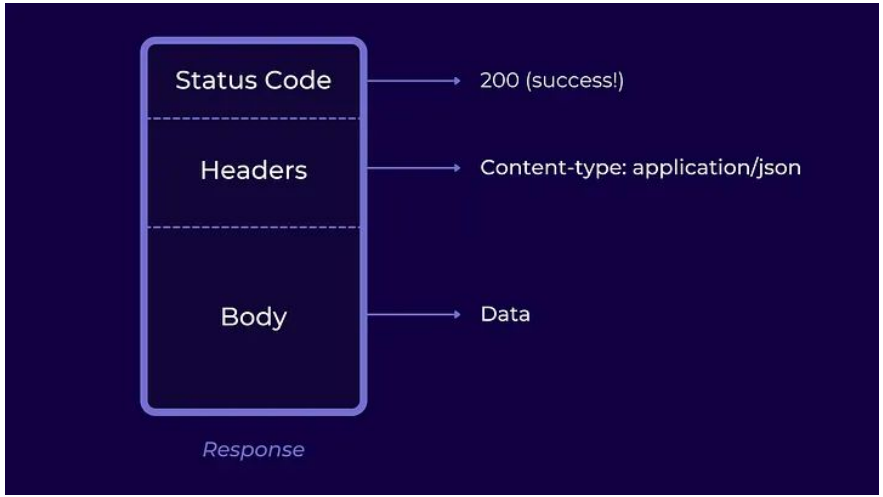
API Architecture: The HTTP Protocol



The structure of HTTP requests

Header Content	SImple Meaning
Host: example.com	Which website you're contacting.
Authorization: Bearer abc123xyz	"Here is my login token. I have already logged in."
Content-Type: application/json	"The data I'm sending is JSON."
User-Agent: Chrome	"I'm using the Chrome browser."

API Architecture: The HTTP Protocol



Response Header	Simple meaning
Content-Type: application/json	"The response is JSON."
Content-Length: 85	"The response body is 85 bytes long."
Set-Cookie: session=abc123xyz	"Store this session cookie and send it back with future requests."

The structure of HTTP responses

Authorisation

Authorisation is a much simpler topic. It's really just about you having appropriate logic to decide what permissions a given authenticated user does or doesn't have.

Summary

- Authentication comes **before** authorisation
- Both are essential for secure systems
- Encryption vs Hashing
  - Encryption = “I want to get the data back later.”
  - Hashing = “I never want to see the original password again.”
- For deeper coverage, UNSW offers additional cybersecurity courses

Feedback

COMP1531 Feedback Survey